TALLINN UNIVERSITY OF TECHNOLOGY

School of Information Technologies

Héctor Barrios Luján 266034IV

WEB-BASED SECURE PASSWORD MANAGER

Web Application Security – Checkpoint 3 (Week 12)

Near-Final State & Draft Project Report (Introduction, Implementation, Security Analysis, and Conclusion)

Supervisor: Ali Ghasempour

Repository: https://github.com/HecBarLu/Web-Application-Security

Tallinn 2026

Checkpoint 3 – Deliverables Compliance Matrix (Week 12)

This document serves as the official Checkpoint 3 submission and near-final project report for the Web-Based Secure Password Manager (Project B) at Tallinn University of Technology. All technical requirements, defensive controls, and automated verification suites are committed and available in the project GitHub repository: https://github.com/HecBarLu/Web-Application-Security.

Deliverable Requirement (Week 12)	Implementation Location	Verification Status
SQL injection prevented via prepared and parameterized statements throughout, verified with injection test payloads.	app.py (sqlite3 '?' placeholders across all queries)	**COMPLETED & VERIFIED (TC-01)**
XSS defences in place (Auto-escaping, URL scheme validation, and strict Content-Security-Policy).	app.py (@app.after_request, is_safe_url) & templates/*.html	**COMPLETED & VERIFIED (TC-02, TC-03)**
Per-user access control enforced on every operation, verified by attempting to read/delete another user's vault entry.	app.py (/vault/view/<id>, /vault/delete/<id> with user_id check)	**COMPLETED & VERIFIED (TC-04, TC-05)**
CSRF protection implemented on all state-changing forms, verified with forged token attempt.	app.py (@app.before_request, secrets.compare_digest) & HTML forms	**COMPLETED & VERIFIED (TC-06)**
Secure error handling and structured logging.	app.py (@app.errorhandler 400/403/404/500, security.log JSON logger)	**COMPLETED & VERIFIED (TC-07)**
Security test cases with results, including attack vectors attempted and how each was mitigated.	test_security.py (Automated unittest suite) & Section 3.3 of Report	**COMPLETED & VERIFIED (5/5 Tests Pass)**
Draft report covering Introduction, Implementation, Security Analysis and Conclusion.	Sections 1, 2, 3, and 4 of this document	**COMPLETED**

1. Introduction

1.1 Background and Motivation

In contemporary web ecosystems, users are routinely required to maintain dozens of distinct digital accounts across financial, academic, corporate, and personal platforms. Because human memory cannot reliably retain unique, high-entropy passwords for each service,

individuals frequently resort to insecure practices such as password reuse, predictable character substitutions, or storing credentials in unencrypted local files. Consequently, credential stuffing, brute-force attacks, and phishing remain primary initial access vectors in modern data breaches. Password managers address this vulnerability by centralizing credential storage inside an encrypted digital vault unlocked by a single high-entropy Master Password.

While commercial password managers provide convenient synchronization across devices, their proprietary codebases often obscure the underlying cryptographic workflows, session management policies, and server-side access controls. This project, developed within the Web Application Security course at Tallinn University of Technology (TalTech), bridges theoretical cybersecurity principles and practical software engineering by designing, building, and verifying a transparent, open-source Web-Based Secure Password Manager.

1.2 Project Scope and Shift-Left Security Philosophy

The core objective of this project is to implement a functional multi-user web credential vault guided by a 'shift-left' security methodology. Rather than treating security as an afterthought or relying on superficial client-side checks, threat modeling and defensive design were integrated from Checkpoint 1 (Architecture and Threat Modeling), translated into working cryptographic and session primitives in Checkpoint 2 (Core Prototype), and hardened with defense-in-depth controls, structured audit logging, and automated exploit verification in Checkpoint 3 (Near-Final State).

- **Confidentiality at Rest and in Transit:** All network traffic is enforced over HTTPS (TLS) using 4096-bit RSA certificates. Stored credentials are encrypted server-side using authenticated symmetric encryption (AES-256-GCM) with keys dynamically derived from the user's Master Password via PBKDF2-HMAC-SHA256, while Master Passwords themselves are verified solely against Argon2id hashes.
- **Strict Isolation and Access Control:** Every database transaction enforces strict per-user ownership checks, preventing horizontal privilege escalation and Insecure Direct Object Reference (IDOR) vulnerabilities.
- **Input Sanitization and Injection Neutralization:** All client-supplied inputs are treated as untrusted. The application enforces server-side regular expression whitelisting, URL scheme validation, parameterized SQL queries, synchronizer CSRF tokens, and strict HTTP Content Security Policies (CSP).
- **Resilience and Auditability:** Global exception handlers suppress sensitive stack traces to prevent information disclosure, while a dedicated JSON security logger records authentication events, access control violations, and input anomalies for forensic traceability.

2. Implementation

2.1 Three-Tier System Architecture and Technology Stack

The application is structured around a strict Three-Tier Architecture that segregates presentation rendering, business and cryptographic logic, and persistent data storage into distinct logical boundaries:

- **Presentation Tier (Client Browser):** Implemented using semantic HTML5, CSS3, and Jinja2 server-rendered templates (login.html, register.html, vault.html, and error.html). The client tier captures user credentials and displays decrypted vault records over an encrypted TLS channel. Because client-side controls (such as HTML maxlength or required attributes) can be trivially bypassed via browser developer tools or intercepting proxies like Burp Suite, the presentation tier is treated strictly as a user-experience layer.
- **Application Tier (Flask Backend Server):** Built on Python 3 and the Flask microframework (app.py). This tier acts as the primary trust boundary and defensive gateway. It enforces session validation, CSRF token verification, input whitelisting, Argon2id authentication, PBKDF2 key derivation, AES-256-GCM encryption/decryption, and structured security logging.
- **Data Tier (SQLite Relational Database):** Persists user account records and encrypted vault items inside an isolated SQLite database file (vault.db). The database stores only Argon2id hashes, cryptographic salts, initialization vectors (IVs), and AES-GCM ciphertexts—never plaintext Master Passwords or plaintext vault secrets.

2.2 Database Schema Design

The persistent storage layer comprises two normalized relational tables initialized automatically at startup via init_db(). The users table stores authentication verifiers and key-derivation parameters, whereas the vault table stores encrypted credential payloads linked via a foreign key constraint (user_id).

```
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,   -- Argon2id PHC string
($argon2id$v=19$m=65536,t=3,p=4$...)
    kdf_salt BLOB NOT NULL         -- 16-byte random cryptographic salt for
PBKDF2HMAC
);

CREATE TABLE IF NOT EXISTS vault (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,      -- Ownership binding to users.id (IDOR
prevention)
```

```
    site_url TEXT NOT NULL,        -- Validated http:// or https:// URL
    site_username TEXT NOT NULL,   -- Account username/email
    encrypted_password BLOB NOT NULL, -- AES-256-GCM ciphertext + 16-byte auth tag
    iv BLOB NOT NULL,              -- 12-byte (96-bit) unique Initialization Vector
    FOREIGN KEY (user_id) REFERENCES users (id)
);
```

2.3 Cryptographic Design and Key Management Workflow

A critical requirement of a password manager is ensuring that a database compromise (such as a stolen backup or file read) does not expose stored credentials in plaintext. To achieve this, the application implements a dual-stage cryptographic architecture using the industry-standard argon2-cffi and cryptography (pyca/cryptography) libraries:

- **1. Master Password Hashing (Argon2id):** When a user registers at POST /register, their Master Password is hashed using Argon2id, the winner of the Password Hashing Competition. Argon2id is a memory-hard and CPU-hard function that resists both GPU cracking arrays and side-channel timing attacks. A separate 16-byte random salt (kdf_salt) is generated via os.urandom(16) specifically for encryption key derivation.
- **2. Dynamic Key Derivation (PBKDF2-HMAC-SHA256):** Upon successful login at POST /login, after Argon2id verifies the Master Password, the server invokes derive_encryption_key(). This function applies PBKDF2HMAC with SHA-256, the user's unique 16-byte kdf_salt, and 200,000 iterations to derive a 256-bit (32-byte) symmetric key. This derived key is held solely in the active encrypted session state and destroyed immediately upon logout or session expiration.
- **3. Authenticated Server-Side Encryption (AES-256-GCM):** When storing a credential at POST /vault/add, the server generates a fresh 12-byte (96-bit) Initialization Vector (IV) via os.urandom(12) and encrypts the target password using AES-256 in Galois/Counter Mode (AESGCM). AES-GCM is an Authenticated Encryption with Associated Data (AEAD) cipher that appends a 128-bit authentication tag to the ciphertext. During decryption at GET /vault or GET /vault/view/<id>, AESGCM verifies this tag before releasing plaintext; if a single bit of the database ciphertext or IV is tampered with, decryption raises an InvalidTag exception and triggers a CRYPTO_INTEGRITY_ERROR log alert.

2.4 Authentication, Session Lifecycle, and TLS Configuration

Session management is hardened against session hijacking, fixation, and cross-site leakage through explicit Flask configuration and request lifecycle hooks:

- **Session Fixation Prevention:** Immediately after verifying a user's Master Password during login, the backend executes session.clear() prior to populating authenticated

session variables. This invalidates any pre-login session identifier an attacker might have planted.

- **Hardened Cookie Attributes:** Session cookies are signed using a 256-bit random secret key (secrets.token_hex(32)) and issued with HttpOnly=True (blocking JavaScript access via document.cookie during XSS attempts), Secure=True (restricting transmission exclusively to encrypted HTTPS connections), and SameSite='Strict' (preventing cookies from being sent on cross-origin requests).
- **Idle Session Timeout:** PERMANENT_SESSION_LIFETIME is set to 15 minutes (timedelta(minutes=15)), refreshed on active requests via @app.before_request, ensuring unattended workstations automatically lock the vault.
- **TLS in Development Setup:** The application server binds to https://127.0.0.1:5000 using a 4096-bit RSA self-signed certificate and private key (certs/cert.pem and certs/key.pem), ensuring end-to-end transport encryption during local operation and testing.

3. Security Analysis and Test Evidence

3.1 Mapping of Implemented Defences to Checkpoint 3 Requirements

In Checkpoint 3, the application was audited against the OWASP Top 10 framework and the specific attack classes studied throughout the course. Below is the detailed analysis of the five core defensive pillars implemented in the codebase:

A. SQL Injection (SQLi) Prevention (OWASP A05 - Injection)

SQL Injection occurs when untrusted user input is concatenated directly into dynamic database queries, allowing attackers to alter SQL syntax, bypass authentication, or exfiltrate entire tables. Throughout app.py, raw string formatting (such as f-strings or % concatenation) is strictly prohibited in SQL statements. Every database interaction—including user registration, login lookup, vault listing, single-item retrieval, insertion, and deletion—exclusively utilizes SQLite prepared and parameterized statements with '?' placeholders:

```
# Parameterized query example from app.py (Login & Per-User Vault Deletion)
user = conn.execute('SELECT * FROM users WHERE username = ?',
(username,)).fetchone()
cursor = conn.execute('DELETE FROM vault WHERE id = ? AND user_id = ?', (item_id,
session['user_id']))
```

The SQLite driver treats bound parameters strictly as literal scalar values rather than executable SQL tokens, rendering tautology payloads (' OR '1'='1' --) and stacked queries ('; DROP TABLE users; --) completely inert.

B. Cross-Site Scripting (XSS) & HTML Injection Defences

In a password manager, a Stored or Reflected XSS vulnerability could allow an attacker to execute arbitrary JavaScript in the victim's browser and scrape decrypted passwords directly from the DOM. To neutralize XSS across all contexts, three independent defensive layers are enforced:

- **1. Context-Aware Output Encoding:** All Jinja2 templates render dynamic variables exclusively via double-brace syntax ({{ item.site_username }}, {{ item.password }}), which automatically converts dangerous characters (<, >, &, ", ') into safe HTML entities (<, >, &, ", ').
- **2. Strict URL Scheme Whitelisting (is_safe_url):** Because vault entries include a clickable website link (<a href="{{ item.site_url }}">), HTML entity encoding alone cannot stop an attacker from submitting a javascript:alert(document.domain) or data: URI scheme. The backend parses every submitted URL with urllib.parse.urlparse() and explicitly verifies parsed.scheme in ('http', 'https') and bool(parsed.netloc) before allowing database insertion.
- **3. Content Security Policy (CSP) and Security Headers:** Every HTTP response is intercepted by @app.after_request to inject restrictive browser headers: Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self'; object-src 'none'; frame-ancestors 'none'; alongside X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and Referrer-Policy: no-referrer. Even if an encoding flaw were ever introduced, the browser's CSP engine blocks inline <script> execution and external script loading.

C. Per-User Access Control & IDOR Prevention (OWASP A01 - Broken Access Control)

Insecure Direct Object Reference (IDOR) occurs when an authenticated user manipulates a resource identifier (such as changing /vault/view/1 to /vault/view/2) to read or delete records belonging to another account. In app.py, every vault route is protected by the @login_required decorator and binds the SQL query directly to both the target item_id and the authenticated session['user_id']:

```
@app.route('/vault/view/<int:item_id>', methods=['GET'])
@login_required
def view_credential(item_id):
    with get_db_connection() as conn:
        row = conn.execute(
            'SELECT * FROM vault WHERE id = ? AND user_id = ?',
            (item_id, session['user_id'])
        ).fetchone()
    if not row:
        log_security_event("IDOR_READ_BLOCKED", f"User id={session['user_id']}
attempted unauthorized read on item_id={item_id}", "WARNING")
        return render_template('error.html', code=403, message="Forbidden: You are
not authorized to view this vault entry."), 403
```

Furthermore, even if a logic bug returned another user's row, the record is encrypted with that user's unique Master Password key, causing AES-GCM decryption to fail mathematically.

D. Cross-Site Request Forgery (CSRF) Protection

In addition to SameSite='Strict' session cookies, the application implements the Synchronizer Token Pattern. A 256-bit cryptographic token (secrets.token_hex(32)) is generated per session and injected as a hidden input field (<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">) in all state-changing forms (/login, /register, /logout, /vault/add, and /vault/delete/<id>). The global @app.before_request hook intercepts every POST request and validates the submitted token using constant-time comparison (secrets.compare_digest), immediately returning HTTP 403 Forbidden and logging a CSRF_VIOLATION if the token is missing or forged.

E. Secure Error Handling and Structured Audit Logging (OWASP A09 & A10)

Unhandled exceptions often leak framework versions, file paths, SQL queries, or Python stack traces to attackers. The application runs with debug=False and registers custom @app.errorhandler hooks for HTTP 400, 403, 404, and generic 500 exceptions, rendering a sanitized error.html view. Simultaneously, the SecurityAudit logger writes JSON-formatted events to security.log capturing the timestamp, severity level, event classification, authenticated user, client IP address, and contextual details:

```
{"timestamp": "2026-04-19 14:22:10,412", "level": "WARNING", "event":
"type=LOGIN_FAILED | user=anonymous | ip=127.0.0.1 | details=Failed authentication
attempt for username 'alice'"}
{"timestamp": "2026-04-19 14:22:11,105", "level": "WARNING", "event":
"type=IDOR_READ_BLOCKED | user=bob_attacker | ip=127.0.0.1 | details=User id=2
attempted unauthorized read on vault item_id=1"}
{"timestamp": "2026-04-19 14:22:11,580", "level": "WARNING", "event":
"type=CSRF_VIOLATION | user=alice | ip=127.0.0.1 | details=Missing or invalid CSRF
token on /vault/add"}
```

3.2 Security Test Cases, Attack Vectors, and Verification Results

To empirically verify every defensive control required by Checkpoint 3, an automated security test suite (test_security.py) built with Python's unittest framework and Flask's test client was executed against the application. Table 1 documents each test case, the exact attack payload attempted, the target route, and the observed mitigation result.

Test ID &	Attack Vector / Payload Attempted	Target Route	Observed Result & Mitigation Mechanism

Category			
TC-01 SQL Injection (Auth Bypass & Stacked Queries)	1) username = ' OR '1'='1' -- 2) username = alice' -- 3) username = admin' OR 1=1# 4) username = '; DROP TABLE users; --	POST /login	MITIGATED (PASS). All 4 payloads failed to bypass login. SQLite parameterized query ('?') bound payloads as literal strings. Generic error returned; LOGIN_FAILED logged.
TC-02 Stored XSS (HTML Script Injection)	site_username = <script>alert(document.cookie)</script>	POST /vault/add & GET /vault	MITIGATED (PASS). Jinja2 auto-escaping encoded payload into <script>alert(document.cookie)</script>. Response also included strict Content-Security-Policy header.
TC-03 Stored XSS (Malicious URI Scheme)	site_url = javascript:alert(document.domain)	POST /vault/add	MITIGATED (PASS). Server-side is_safe_url() rejected non-HTTP/HTTPS scheme before DB insertion. Flash error displayed and UNSAFE_URL_BLOCKED event logged.
TC-04 Broken Access Control (IDOR Cross-User Read)	User 'alice' (id=1) created vault item #1. Authenticated as 'bob_attacker' (id=2) and requested GET /vault/view/1.	GET /vault/view/1	MITIGATED (PASS). Server returned HTTP 403 Forbidden. SQL query 'WHERE id = ? AND user_id = ?' returned empty set. Alice's secret was never exposed; IDOR_READ_BLOCKED logged.
TC-05 Broken Access Control (IDOR Cross-User Delete)	Authenticated as 'bob_attacker' (id=2) and submitted POST /vault/delete/1 targeting Alice's vault entry #1.	POST /vault/delete/1	MITIGATED (PASS). Server returned HTTP 403 Forbidden (cursor.rowcount == 0). Alice's record remained intact in DB; IDOR_DELETE_BLOCKED logged.
TC-06 CSRF Protection (Forged State-Changing POST)	Submitted POST /vault/add with forged token: csrf_token = forged_invalid_token_0000	POST /vault/add	MITIGATED (PASS). Intercepted by @app.before_request. Constant-time comparison failed, returning HTTP 403 Forbidden ('CSRF verification failed') and logging CSRF_VIOLATION.
TC-07	Requested non-existent endpoint	GET	MITIGATED (PASS).

Error Handling & Structured Logging	GET /non_existent_route_12345 and inspected response body & security.log.	/non_existent_route_12345	Custom error handler returned HTTP 404 with generic error.html template. Zero Python/Werkzeug stack traces leaked; security.log verified on disk.

3.3 Automated Security Test Suite Execution Output

Running python test_security.py on a clean environment executes all five comprehensive test methods covering TC-01 through TC-07 and confirms 100% pass rate:

```
$ python test_security.py
test_01_sql_injection_mitigation (__main__.SecurityVerificationTestSuite)
TC-01: Verify SQL Injection payloads fail to bypass authentication or corrupt
database. ... ok
test_02_xss_defense_and_scheme_validation (__main__.SecurityVerificationTestSuite)
TC-02: Verify Stored XSS payloads are HTML-encoded and javascript: URLs are
rejected. ... ok
test_03_per_user_access_control_idor_prevention
(__main__.SecurityVerificationTestSuite)
TC-03: Verify User B cannot read or delete User A's vault entry (IDOR Prevention).
... ok
test_04_csrf_protection_enforcement (__main__.SecurityVerificationTestSuite)
TC-04: Verify state-changing POST requests without valid CSRF token are blocked with
403. ... ok
test_05_secure_error_handling_and_logging (__main__.SecurityVerificationTestSuite)
TC-05: Verify errors do not leak stack traces and security events are logged. ... ok

----------------------------------------------------------------------
Ran 5 tests in 1.842s

OK
```

4. Conclusion

The development of the Web-Based Secure Password Manager across Checkpoints 1, 2, and 3 demonstrates how foundational web application security principles can be systematically engineered into a working software product. By adopting a Three-Tier Architecture and assuming that all client-supplied data is inherently untrusted, the application enforces security at every layer of execution.

Cryptographically, combining Argon2id for Master Password verification with PBKDF2-HMAC-SHA256 key derivation and AES-256-GCM authenticated encryption ensures strong confidentiality and tamper detection for stored vault records. Architecturally, the integration

of parameterized SQLite queries, context-aware Jinja2 output encoding, strict URL scheme validation, Content Security Policy headers, synchronizer CSRF tokens, and explicit per-user ownership checks neutralizes critical vulnerability classes including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and Insecure Direct Object References (IDOR). Finally, custom exception handlers and structured JSON security logging provide operational resilience and auditability without leaking internal implementation details. All Checkpoint 3 deliverables have been implemented, verified through automated exploit testing, and published to the project repository.

Appendix A – Clean Machine Setup & Verification Instructions

To reproduce the application setup and execute the automated security verification suite on a clean machine, run the following commands in Git Bash or a POSIX-compatible terminal:

```
# 1. Clone the repository and initialize virtual environment
git clone https://github.com/HecBarLu/Web-Application-Security.git
cd Web-Application-Security
python -m venv venv
source venv/Scripts/activate   # On Linux/macOS: source venv/bin/activate
pip install -r requirements.txt

# 2. Generate 4096-bit RSA self-signed TLS certificates
unset OPENSSL_CONF
mkdir -p certs
MSYS_NO_PATHCONV=1 openssl req -x509 -newkey rsa:4096 -nodes -out certs/cert.pem -
keyout certs/key.pem -days 365 -subj "/CN=localhost"

# 3. Execute automated security test suite (Checkpoint 3 verification)
python test_security.py

# 4. Launch HTTPS server
python app.py
# Access in browser: https://127.0.0.1:5000
```